Load Balancers are Not Magic: Atlassian Cloud Outage

Source: Arpit Bhayani Dissection • Cloud Reliability
Focus: Managed ELB Mechanics, Cold Starts & Scaling Limits

1. The "Infinite Black Box" Fallacy

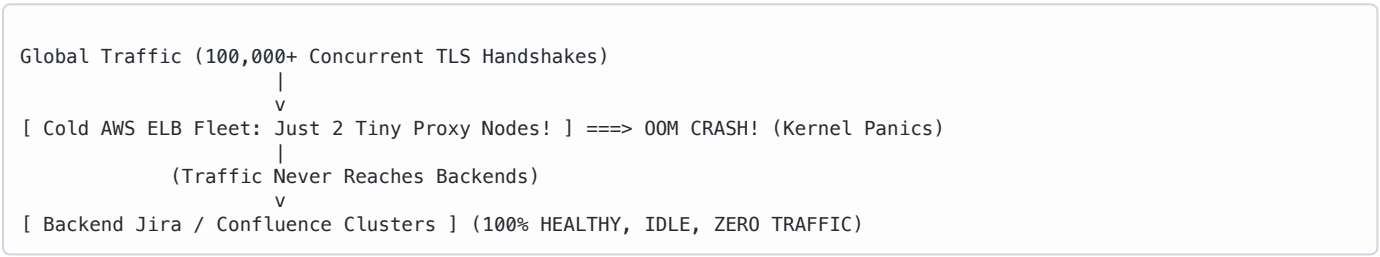
In cloud system design, software engineers frequently treat managed load balancers (such as AWS Application Load Balancers or Classic ELBs) as infinite, flawless abstractions that automatically absorb unlimited traffic surges without friction.

Reality: Load balancers are not magic; they are fleets of standard Linux proxy servers (typically running Envoy or Nginx on EC2 instances). They have finite physical memory, CPU cores, Linux socket buffers, connection state tables, and network interfaces. When subjected to sudden, un-warmed traffic surges, they can exhaust memory, trigger Out-Of-Memory (OOM) kernel panics, and crash completely.

2. The Atlassian Incident: Symptoms & Paradox

In 2019, Atlassian suffered a severe cloud outage characterized by a baffling operational paradox:

- **Backend Pods Were 100% Healthy:** Jira and Confluence backend microservices displayed zero errors, low CPU, normal memory, and healthy health checks.
- **Zero Ingress Traffic:** Client traffic reaching the application pods flatlined to zero. Requests were dying before ever making it through the front door.
- **User Blackout:** Millions of global users were greeted with connection resets and HTTP 504 Gateway Timeouts.



> **The Scale-Out Lag:** AWS ELB scales horizontally by monitoring traffic curves and provisioning additional proxy instances. However, scaling takes between **3 and 10 minutes**. An instantaneous 100x traffic surge easily destroys cold proxy instances before auto-scaling can respond.

The Cold Deployment Trap & Proxy Memory Exhaustion

Source: Arpit Bhayani Dissection • Cloud Reliability
Focus: Recreating ELBs, TLS Handshakes & OOM Killer

3. The Cold Deployment Anti-Pattern

The root cause of Atlassian's outage was an automated deployment script that recreated the entire infrastructure stack—including the load balancer—on blue/green deployments:

```
OLD FLAWED DEPLOYMENT FLOW:
1. CI/CD pipeline creates "Green" environment with brand-new, un-warmed ELB.
2. New ELB initializes with default baseline: 2 small proxy nodes.
3. Pipeline updates Route 53 DNS to instantly swing 100% of global traffic to new ELB!
4. Massive traffic collides with 2 cold proxy nodes -> Instant OOM Crash.
```

4. The Anatomy of Proxy Memory Exhaustion (OOM)

When hundreds of thousands of concurrent connections hit the two small proxy instances simultaneously:

- **Cryptographic TLS Storm:** Asymmetric RSA/ECDHE key exchanges saturated 100% of CPU cores.
- **Socket Buffer Flooding:** Every TCP connection allocates kernel read and write buffers (`rmem`/`wmem`, 16–128 KB each). Thousands of incoming connections rapidly consumed all physical DRAM.
- **State Table Overflow:** In-memory connection tracking tables exhausted available heap space.
- **The Linux OOM Killer:** The OS kernel invoked `oom-killer` to preserve system stability, killing the proxy processes. The proxies crashed and entered an infinite crash-restart loop.

> **Core Axiom:** The primary failure was not application software, but the failure to account for the physical resource limits and provisioning latency of managed cloud networking infrastructure.

Mitigations, Target Shifting & System Takeaways

5. Production Mitigation & Guardrails

Modern high-availability infrastructure adheres to three foundational rules:

A. Shift Targets, Never Recreate Load Balancers

Treat the load balancer fleet as durable, persistent infrastructure:

```
CORRECT ARCHITECTURE:
[ Persistent, Pre-Warmed Load Balancer Fleet (30+ Proxy Nodes) ]
                  |
                  +---> Target Group Blue: Version 1.0 (Drain traffic)
                  +---> Target Group Green: Version 2.0 (Shift traffic dynamically)
```

B. Pre-Warming for Major Traffic Events

Before product launches or planned marketing campaigns, submit pre-warming tickets to cloud providers. Engineers manually scale the proxy fleet to dozens of high-memory instances in advance.

C. Gradual Weighted DNS Ramp-Up

If DNS cutovers cannot be avoided, ramp traffic gradually: $10\% o 25\% o 50\% o 100\%$ over 20+ minutes to allow proxy auto-scalers to expand capacity organically.

6. First-Principles System Design Takeaways

Architecture Dimension	Common Misconception	Production Reality
Cloud Load Balancers	Infinite black boxes that scale instantly	Finite fleets of real servers with CPU/RAM limits and 5-minute auto-scale lag
Deployments	Spinning up new load balancers per deploy	Keep warmed load balancers static; shift backend target groups
Traffic Shifting	Instantaneous 0% to 100% DNS flips	Gradual weighted percentage ramp-ups over 15–30 minutes
Surge Handling	Rely on cloud auto-scaling during spikes	Proactively pre-warm proxy infrastructure ahead of large surges